Project work, part 4 - Machine Learning -2

Tasks

Jupyter Notebook

I saw that production data has 3,647 fewer rows than consumption data.
I padded the missing data with NaN values (specifically for NO5 and wind, covering 2021-01-01 to 2021-06-01).

In [2]:
```
from utils.data_loaders import load_mongoDB

MONGO_DATABASE = "elhub_data"
MONGO_COLLECTION_PRODUCTION_FULL = "production_data_2021_2024"
MONGO_COLLECTION_CONSUMPTION_FULL = "consumption_data_2021_2024"

df_consumption = load_mongoDB(MONGO_COLLECTION_CONSUMPTION_FULL, MONGO_DATABASE)
df_consumption.head()
df_consumption.tail()
```

```
2025-11-19 07:22:06.042 WARNING streamlit.runtime.caching.cache_data_api: No runtime found, using MemoryCacheStorageManager
2025-11-19 07:22:06.042 WARNING streamlit.runtime.caching.cache_data_api: No runtime found, using MemoryCacheStorageManager
2025-11-19 07:22:06.047 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:06.219
  Warning: to view this Streamlit app on a browser, run it with the following
  command:

    streamlit run c:\Users\oriek\miniconda3\envs\D2D_env\lib\site-packages\ipykernel_launcher.py [ARGUMENTS]
2025-11-19 07:22:06.220 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:06.222 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:06.222 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:06.789 Thread 'Thread-3': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:06.804 Thread 'Thread-3': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:06.804 Thread 'Thread-3': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:39.623 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:39.642 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:39.644 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

Out[2]:

	pricearea	datatype	groupname	starttime	quantitykwh
876595	NO5	Consumption	cabin	2024-04-29T09:00:00+02:00	31118.297
876596	NO5	Consumption	household	2024-10-26T22:00:00+02:00	390373.800
876597	NO5	Consumption	secondary	2024-04-20T14:00:00+02:00	1031581.400
876598	NO5	Consumption	household	2024-03-14T08:00:00+01:00	485459.100
876599	NO5	Consumption	tertiary	2023-09-24T10:00:00+02:00	234220.220

In [3]:

```
df_production = load_mongoDB(MONGO_COLLECTION_PRODUCTION_FULL, MONGO_DATABASE)
df_production.head()
df_production.tail()
```

```
2025-11-19 07:22:42.915 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:42.916 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:42.916 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:42.917 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:43.426 Thread 'Thread-4': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:43.426 Thread 'Thread-4': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:22:43.429 Thread 'Thread-4': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:23:28.305 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:23:28.306 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-19 07:23:28.306 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

Out[3]:

	pricearea	datatype	groupname	starttime	quantitykwh
872948	NO5	Production	solar	2023-10-14T08:00:00+02:00	10.396
872949	NO5	Production	solar	2024-11-25T12:00:00+01:00	189.010
872950	NO5	Production	solar	2024-04-29T02:00:00+02:00	88.424
872951	NO5	Production	thermal	2024-06-15T09:00:00+02:00	15172.930
872952	NO5	Production	thermal	2024-05-28T16:00:00+02:00	18604.510

In [4]:

```python
cons_per_time = df_consumption.groupby("starttime").size()
prod_per_time = df_production.groupby("starttime").size()

cons_per_time.describe(), prod_per_time.describe()
```

Out[4]:

```
(count    35064.0
 mean        25.0
 std          0.0
 min         25.0
 25%         25.0
 50%         25.0
 75%         25.0
 max         25.0
 dtype: float64,
 count    35064.000000
 mean        24.895990
 std          0.305278
 min         24.000000
 25%         25.000000
 50%         25.000000
 75%         25.000000
 max         25.000000
 dtype: float64)
```

In [21]:
```
actual_per_group = (
    df_production[df_production["pricearea"] == "NO5"]
    .groupby("groupname")
    .size()
)

missing_per_group = expected_per_group - actual_per_group
missing_per_group

summary = pd.DataFrame({
    "expected_rows": expected_per_group,
    "actual_rows": actual_per_group,
    "missing_rows": missing_per_group,
    "missing_percent": (missing_per_group / expected_per_group * 100).round(3)
})

print(summary.sort_values("missing_rows", ascending=False))
```

```
           expected_rows  actual_rows  missing_rows  missing_percent
groupname
wind               35064        31417          3647           10.401
hydro              35064        35064             0            0.000
other              35064        35064             0            0.000
solar              35064        35064             0            0.000
thermal            35064        35064             0            0.000
```

In [ ]:
```
import pandas as pd

full_index = pd.date_range(
    start="2021-01-01 00:00:00",
    end="2024-12-31 23:00:00",
    freq="h",
    tz="Europe/Oslo"
)

template = pd.DataFrame({"starttime": full_index})

df_no5_wind = df_production[
    (df_production["pricearea"] == "NO5") &
    (df_production["groupname"] == "wind")
].copy()

df_no5_wind["starttime"] = pd.to_datetime(df_no5_wind["starttime"], utc=True)
df_no5_wind["starttime"] = df_no5_wind["starttime"].dt.tz_convert("Europe/Oslo")

df_no5_wind_full = template.merge(
    df_no5_wind,
    on="starttime",
    how="left"
)

df_no5_wind_full["pricearea"] = "NO5"
df_no5_wind_full["datatype"]  = "production"
df_no5_wind_full["groupname"] = "wind"

df_production_filled = pd.concat([
```

```
    df_production[
        ~(
            (df_production["pricearea"] == "NO5") &
            (df_production["groupname"] == "wind")
        )
    ],  # all other data unchanged
    df_no5_wind_full
])

# df_production_filled = df_production_filled.sort_values("starttime")

print(df_production_filled)
print(len(df_production_filled))
```

```
      pricearea    datatype groupname                  starttime  quantitykwh
0           NO1  Production     hydro  2022-01-19T02:00:00+01:00  1500431.000
1           NO1  Production     hydro  2021-09-11T21:00:00+02:00  1868775.200
2           NO1  Production     hydro  2021-10-26T02:00:00+02:00  2118256.200
3           NO1  Production     other  2021-06-09T08:00:00+02:00        6.000
4           NO1  Production      wind  2021-05-10T02:00:00+02:00    48078.945
...         ...         ...       ...                        ...          ...
35059       NO5  production      wind  2024-12-31 19:00:00+01:00        0.000
35060       NO5  production      wind  2024-12-31 20:00:00+01:00        0.000
35061       NO5  production      wind  2024-12-31 21:00:00+01:00        0.000
35062       NO5  production      wind  2024-12-31 22:00:00+01:00        0.000
35063       NO5  production      wind  2024-12-31 23:00:00+01:00        0.000

[876600 rows x 5 columns]
876600
```

Streamlit app

Refactoring

- Plotting
  - Exchanged static plots (matplotlib) with dynamic plots (Plotly)
- Structure and navigation
  - On the start page, the user must first select an area and the type of energy data to analyze (e.g., Production or Consumption). To ensure consistency across all dependent visualizations and analyses, the user is not allowed to reselect these primary parameters once chosen. A change in the selected area or data type requires the user to explicitly click the "Reset" button located on the top of the page.
  - All application modules have independent date selection.

Observation Meteorology and energy production

- After implementation, play with the controls and check if you can spot any changes incorrelations in normal conditions and in/after extreme weather events.
- Between Wednesday, January 31, and Thursday, February 1, 2024, extreme weather event Ingunn impacted Trøndelag, bringing extremely strong winds. (I believe I was at

home during these days; my commuter boat was cancelled.) While the attached PDF file (Ingunn and wind.pdf) shows that wind speeds were high, the production of wind energy did not increase proportionally. This pattern was observed not only during these specific days but also in the following days. The System Wind Capacity (SWC) was zero or negative

Bonus content

I have tried to implement the following contents:

- Waiting time
  - Use progress bars, spinners, or similar to indicate work in progress.
  - Cache everything that is possible to cache.
- Error handling
  - Try to incorporate checks in the app that handle missing data connections (API and database) and NaN/missing values in the data.
  - Catch the errors and give useful feedback instead of crashing or giving cryptic errormessages.
- Forecasting
  - Add weather properties to the list of exogenous variables and download when needed.

Log Describing the Compulsory Work

Coding in Jupyter Notebook

- I used the code from previous projects, and it went relatively smoothly.
- As I mentioned in the beginning of the report, I created/modified some functions to handle both production and consumption data, and a new table/data structure to accommodate the new functions.

Coding in Streamlit app

1. Exchange matplotlib figures with dynamic plots by plotly
   With the help of AI tools, I exchanged all the matplotlib figures for plotly plots in the Jupyter Notebook environment before merging the code into the Streamlit app.

2. All new functions
   I modified all new functions, including machine learning (lagged_correlation_plot), SARIMAX, map_folium_choropleth, and others. All the new functions were tested in the Jupyter Notebook environment before merging them into the Streamlit app.

3. Structure and navigation before and under coding
   I thought about the necessary input and output, what the user should or should not select, the ranges the user selection covers, and the user experience (e.g., speed,

intuitiveness). Since I did not have an overview and understanding of what Streamlit could provide me/us, it was challenging to determine what I could serve the users and how to tune the Streamlit app.

4. Coding, testing, debugging of the entire project
   I have one start/home page, seven subpages, and twelve utility files (functions, etc.). It is easy to lose track of the overview. Small changes in one file can have big consequences in other files.

5. Access to remote MongoDB Atlas has become slow
   As I mentioned, this happened suddenly. After testing the connections with some help and advice from AI tools, I decided to install MongoDB locally on my PC to continue developing the Streamlit app.

SARIMAX

- I noticed that the predictions for physical quantities (energy production, energy consumption, wind speed, etc.) included negative numbers. As these variables cannot naturally be negative, I implemented conditional code to enforce non-negativity for all forecasts, excluding temperature.

- Calculating hourly data for four years is time-consuming. Therefore, I added a selection box for the datetime index to allow the user to choose a shorter period. This focused selection enabled the creation of many interesting and informative plots.

- My function, sarimax.py, utilizes both energy data and weather data. While some of the resulting plots look acceptable, others are erratic (or unreliable). It has proven too complicated for me to tune (or optimize) the variables (model parameters, exogenous factors, and orders) effectively so that all the resulting plots are reliable and understandable.

GidHub

I pushed files to GitHub, but the Streamlit app does not work online or, if it does, it runs extremely slowly.

Brief Description of AI Usage

I used AI tools to generate initial versions of the code, to refine and customize them, and to debug errors.

Coding and Debugging

I described my requirements, and AI tools suggested initial drafts. While some of the generated code worked well, others required some modification.

Structure and navigation, testing, debugging of the entire project

This was the most challenging area in this project. When an error occurred, AI tools would tell me where and how I should modify the code. As soon as I made the change, other parts of the project would stop working. This happened repeatedly.

I used AI tools to check the overall consistency of the entire project. The tools cleaned up my code by: checking for lines defined twice, ensuring the use of st.session_state was consistent throughout the project, and suggesting the creation of new functions.

When I did not inform the AI tools of my own modifications, they tended to stay within their own understanding and context, without incorporating my ideas and changes. AI tools clearly require context to answer me effectively. The amount of detail regarding my ideas and activities that should be shared with the AI tools remains a key question.